

数据结构与算法

第2章 线性表

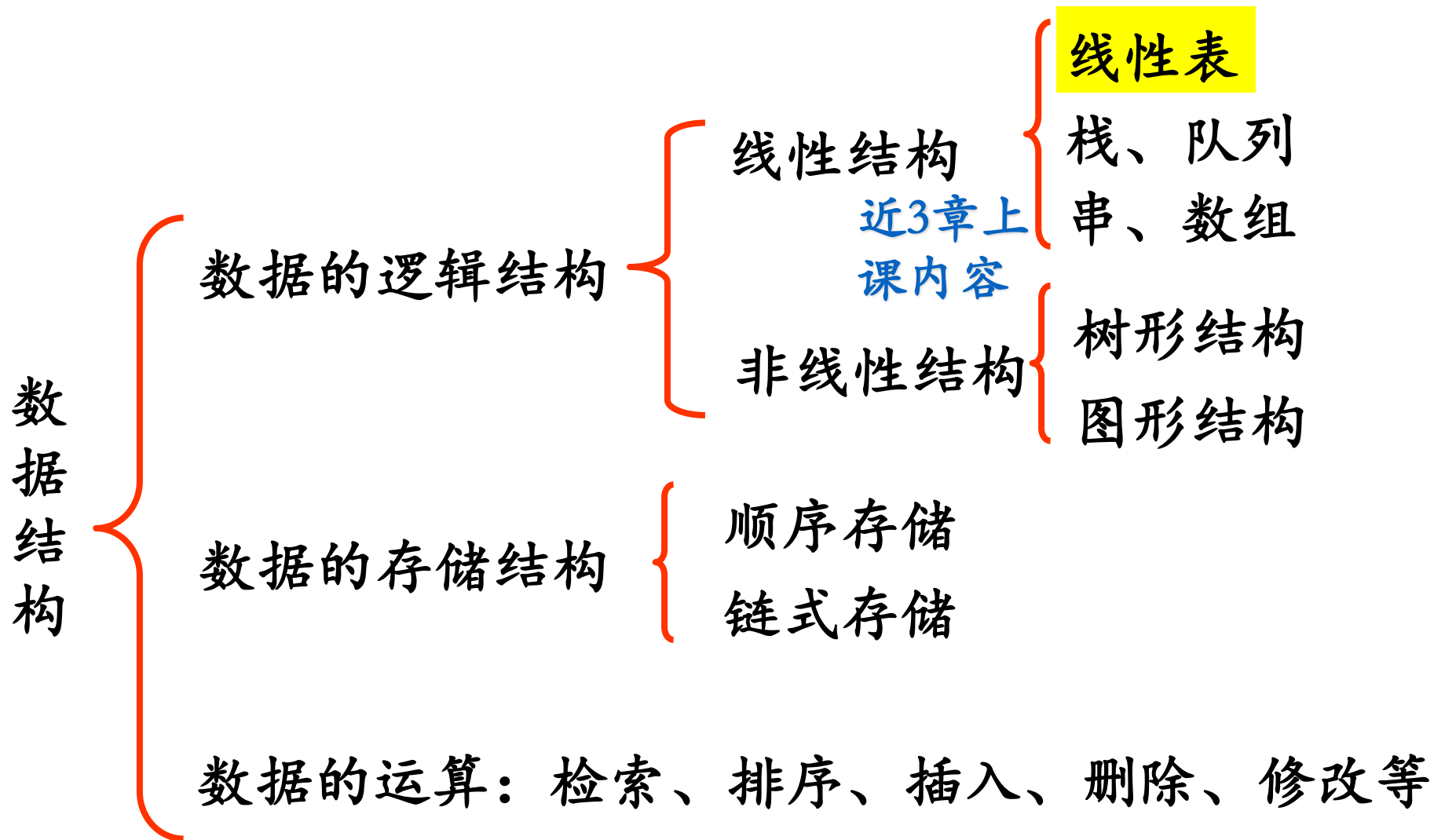
张丽淼

光电信息获取与防护技术全国重点实验室

zhanglm@ahu.edu.cn

2026年3月11日

数据结构 (Data Structure) 是相互之间存在一种或多种特定关系的数据元素的集合。



线性结构的定义:

若结构是非空有限集, 则有且仅有一个开始结点和一个终端结点, 并且所有结点都最多只有一个直接前趋和一个直接后继。

可表示为: $(a_1, a_2, \dots, a_n)$

线性结构表达式： $(a_1, a_2, \dots, a_n)$

线性结构的特点：

- ① 只有一个首结点和尾结点；
- ② 除首尾结点外，其他结点只有一个直接前驱和一个直接后继。

简言之，线性结构反映结点间的逻辑关系是一对一的

线性结构包括线性表、堆栈、队列、字符串、数组等等，其中，最典型、最常用的是



线性表

第2章 线性表



教学目标

1. 了解线性结构的特点
2. 掌握顺序表的定义、查找、插入和删除
3. 掌握链表的定义、创建、查找、插入和删除
4. 能够从时间和空间复杂度的角度比较两种存储结构的不同特点及其适用场合

教学内容

2.1 线性表的定义和特点

2.2 案例引入

2.3 线性的类型定义

2.4 线性表的顺序表示和实现

2.5 线性表的链式表示和实现

2.6 顺序表和链表的比较

2.7 线性表的应用

2.8 案例分析与实现



2.1 线性表的定义和特点

线性表的定义：

具有相同特性的数据元素的有限序列

$(a_1, a_2, \dots, a_{i-1}, a_i, a_{i+1}, \dots, a_n)$



2.1 线性表的定义和特点

线性表的定义：

具有相同特性的数据元素的**有限序列**

$(a_1, a_2, \dots, a_{i-1}, a_i, a_{i+1}, \dots, a_n)$

数据元素

线性起点

(起始结点、

表头元素)

a_i 的直接前趋

a_i 的直接后继

线性终点

(终端结点、

表尾元素)

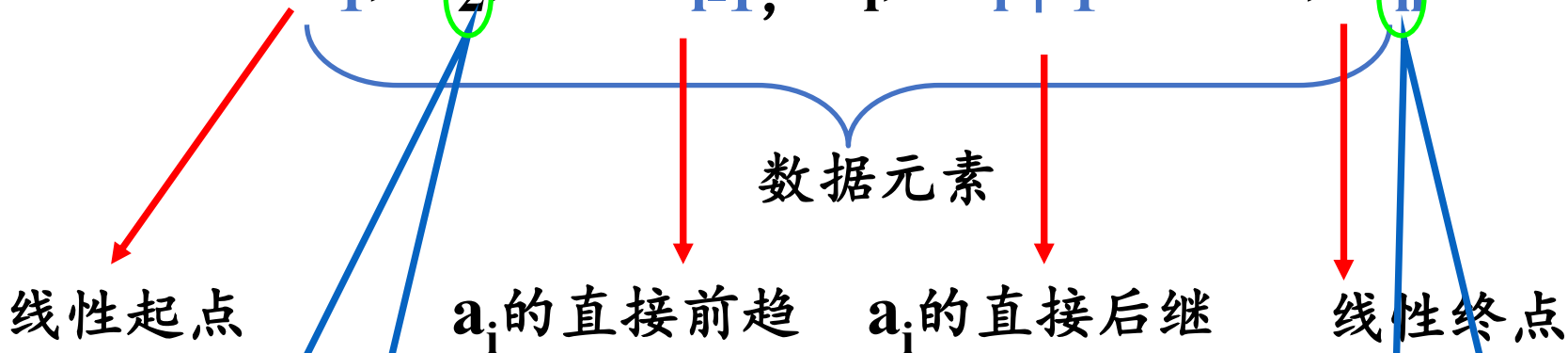


2.1 线性表的定义和特点

线性表的定义：

具有相同特性的数据元素的有限序列

$(a_1, a_2, \dots, a_{i-1}, a_i, a_{i+1}, \dots, a_n)$



下标，是元素的序号，表示元素在表中的位置

$n=0$ 时称为 空表

n 为元素总个数，即表长

例1 分析26个英文字母组成的英文表

(A, B, C, D, , Z)

数据元素都是字母; 元素间关系是线性

例1 分析26个英文字母组成的英文表

(A, B, C, D,, Z)

数据元素都是字母； 元素间关系是线性

例2 分析学生情况登记表

学号	姓名	性别	年龄	班级
041810205	于春梅	女	18	04级计算机1班
041810260	何仕鹏	男	20	04级计算机2班
041810284	王 爽	女	19	04级计算机3班
041810360	王亚武	男	18	04级计算机4班
:	:	:	:	:

数据元素都是记录； 元素间关系是线性

同一线性表中的元素必定具有相同特性、线性关系

线性表的逻辑特征：

- 在非空的线性表中，有且仅有一个开始结点 a_1 ，它没有直接前驱，仅有一个直接后继 a_2 ；
- 在非空的线性表中，有且仅有一个终端结点 a_n ，它没有直接后继，仅有一个直接前驱 a_{n-1} ；
- 其余的内部结点 $a_i (2 \leq i \leq n-1)$ 都有且仅有一个直接前驱 a_{i-1} 和一个直接后继 a_{i+1} 。

线性表是一种典型的线性结构。



2.2 案例引入

案例2.1：一元多项式的表示及其运算

$$P_n(x) = p_0 + p_1x + p_2x^2 + \dots + p_nx^n$$

主要运算：多项式相加、相减、相乘等

如何用程序设计语言来表示以及实现相应的操作？



2.2 案例引入

案例2.1：一元多项式的表示及其运算

$$P_n(x) = p_0 + p_1x + p_2x^2 + \dots + p_nx^n$$

主要运算：多项式相加、相减、相乘等

【分析】如何表示多项式？

多项式的关键数据：

- 多项式的项数 **n**
- 各项系数 **p_i** 以及指数 **i**



2.2 案例引入

方法1:顺序存储结构直接表示

$$P_n(x) = p_0 + p_1x + p_2x^2 + \dots + p_nx^n$$

线性表 $P = (p_0, p_1, p_2, \dots, p_n)$

$$P(x) = 10 + 5x - 4x^2 + 3x^3 + 2x^4$$

数组表示

(每一项的指数*i*隐含在其系数 p_i 的序号中)

指数 (下标 <i>i</i>)	0	1	2	3	4
系数 $p[i]$	10	5	-4	3	2



2.2 案例引入

方法1:顺序存储结构直接表示

$$P_n(x) = p_0 + p_1x + p_2x^2 + \dots + p_nx^n$$

线性表 $P = (p_0, p_1, p_2, \dots, p_n)$

$$P(x) = 10 + 5x - 4x^2 + 3x^3 + 2x^4$$

数组表示

(每一项的指数*i*隐含在其系数 p_i 的序号中)

指数 (下标 <i>i</i>)	0	1	2	3	4
系数 $p[i]$	10	5	-4	3	2

两个多项式相加：两个数组对应的分量相加

$$R_n(x) = P_n(x) + Q_m(x)$$



线性表 $R = (p_0 + q_0, p_1 + q_1, p_2 + q_2, \dots, p_m + q_m, p_{m+1}, \dots, p_n)$

$$R_n(x) = P_n(x) + Q_m(x)$$



线性表 $R = (p_0 + q_0, p_1 + q_1, p_2 + q_2, \dots, p_m + q_m, p_{m+1}, \dots, p_n)$

稀疏多项式

$$S(x) = 1 + 3x^{10000} + 2x^{20000}$$



方法2:顺序存储结构表示非零项

$$P_n(x) = p_1 x^{e_1} + p_2 x^{e_2} + \dots + p_m x^{e_m}$$


线性表 $P = ((p_1, e_1), (p_2, e_2), \dots, (p_m, e_m))$

多项式**非零项**的数组表示

(a) $A(x) = 7 + 3x + 9x^8 + 5x^{17}$

下标i	0	1	2	3
系数a[i]	7	3	9	5
指数	0	1	8	17

(b) $B(x) = 8x + 22x^7 - 9x^8$

下标i	0	1	2
系数b[i]	8	22	-9
指数	1	7	8

方法2:顺序存储结构表示非零项

$$P_n(x) = p_1 x^{e_1} + p_2 x^{e_2} + \dots + p_m x^{e_m} \rightarrow$$

线性表 $P = ((p_1, e_1), (p_2, e_2), \dots, (p_m, e_m))$

多项式**非零项**的数组表示

(a) $A(x) = 7 + 3x + 9x^8 + 5x^{17}$

下标i	0	1	2	3
系数a[i]	7	3	9	5
指数	0	1	8	17

(b) $B(x) = 8x + 22x^7 - 9x^8$

下标i	0	1	2
系数b[i]	8	22	-9
指数	1	7	8

线性表 $A = ((7, 0), (3, 1), (9, 8), (5, 17))$

线性表 $B = ((8, 1), (22, 7), (-9, 8))$

求两个多项式的和：

线性表A= ((7, 0), (3, 1), (9, 8), (5, 17))

线性表B= ((8, 1), (22, 7), (-9, 8))



- 创建一个**新数组c**

按指数大小有序排列！

- 分别从头遍历比较a和b的每一项

- ✓ **指数相同**，对应系数相加，若其和不为零，则在c中增加一个新项

- ✓ **指数不相同**，则将指数较小的项复制到c中

- 一个多项式已遍历**完毕**时，将另一个剩余项依次复制到c中即可

●顺序存储结构存在问题

- ✓存储空间分配不灵活
- ✓运算的空间复杂度高

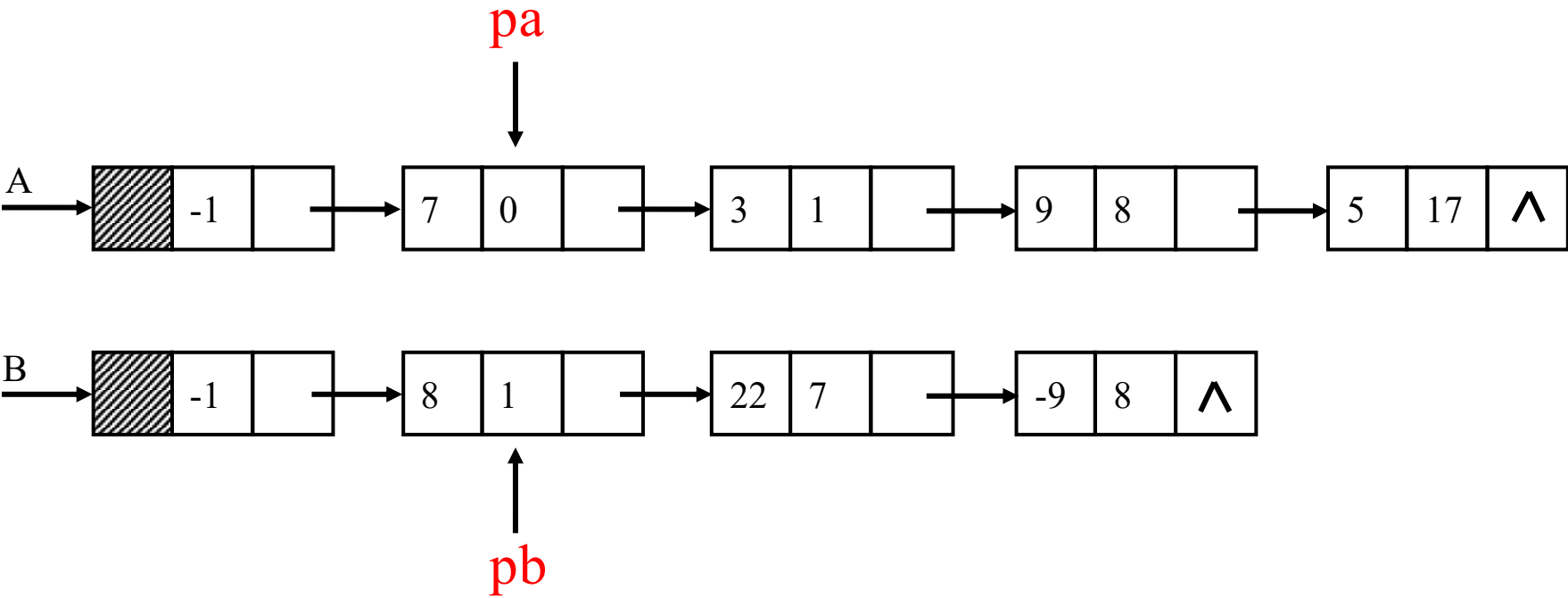


方法3:链表结构存储非零项

$A_{17}(x)=7+3x+9x^8+5x^{17}$

$B_8(x)=8x+22x^7-9x^8$

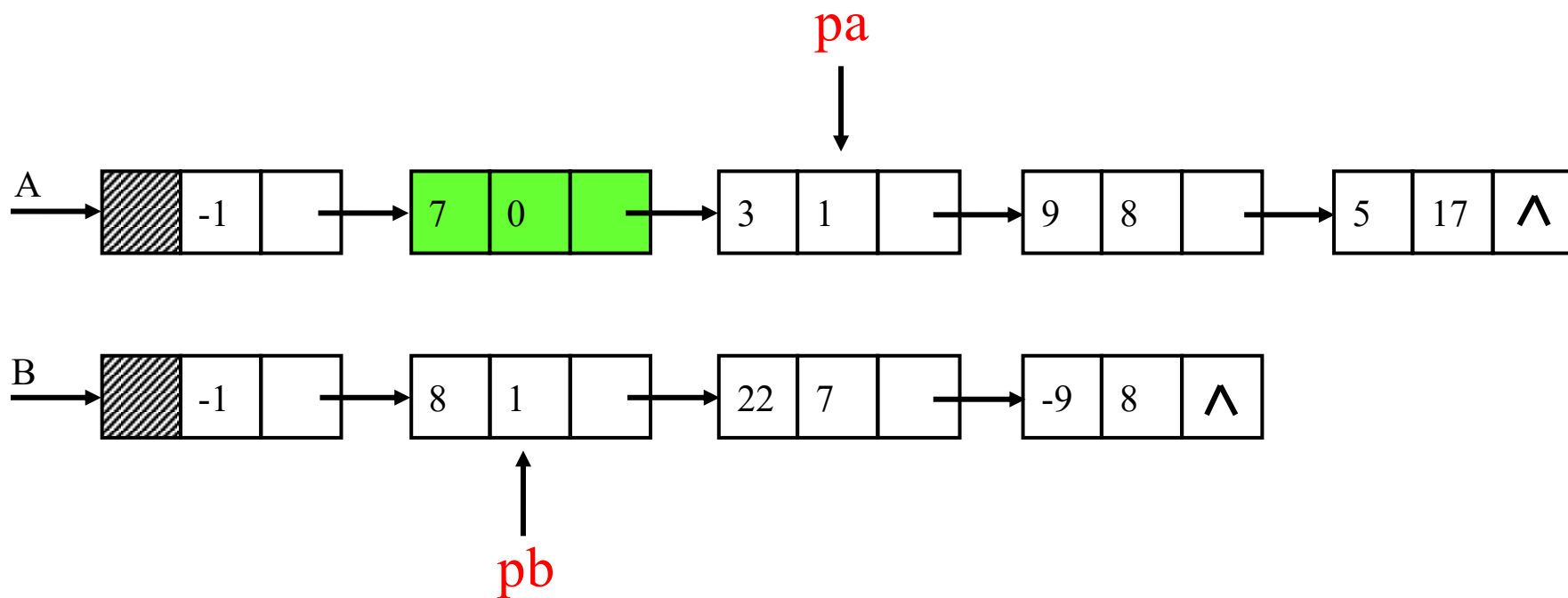
coef	expo	link
------	------	------



多项式相加

$$A_{17}(x)=7+3x+9x^8+5x^{17}$$

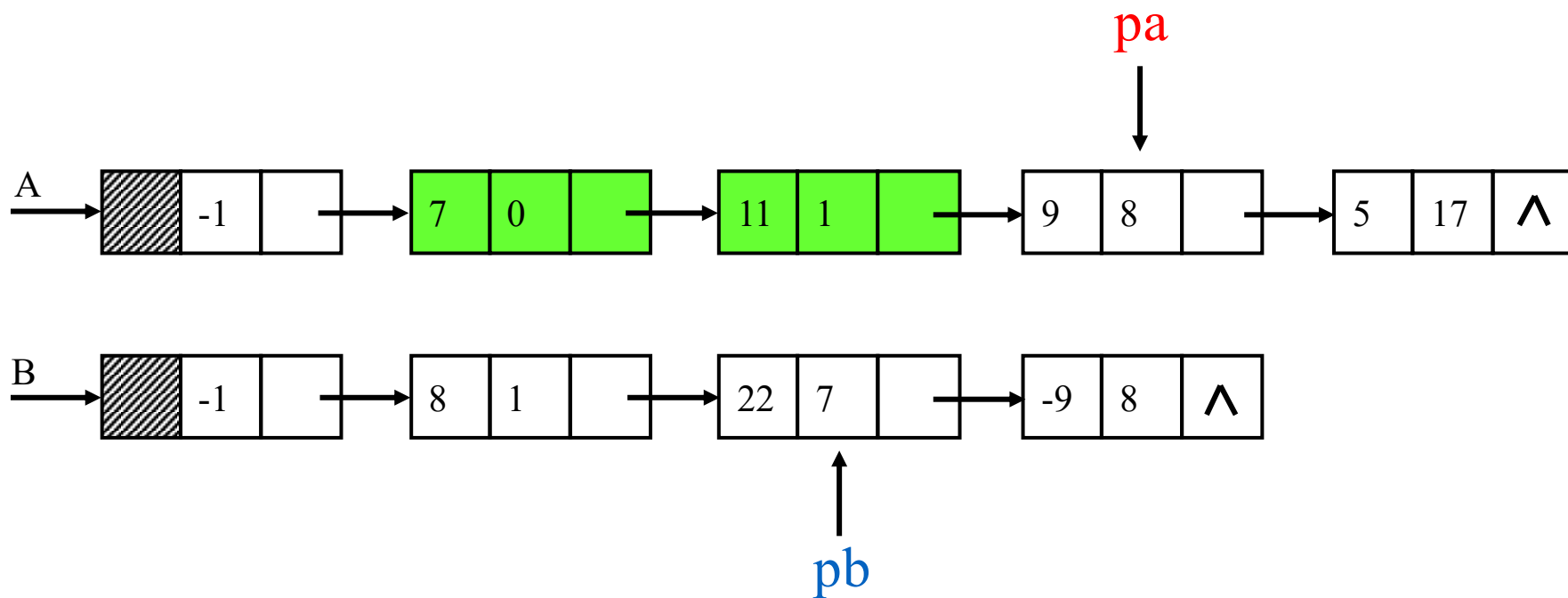
$$B_8(x)=8x+22x^7-9x^8$$



多项式相加

$$A_{17}(x) = 7 + 3x + 9x^8 + 5x^{17}$$

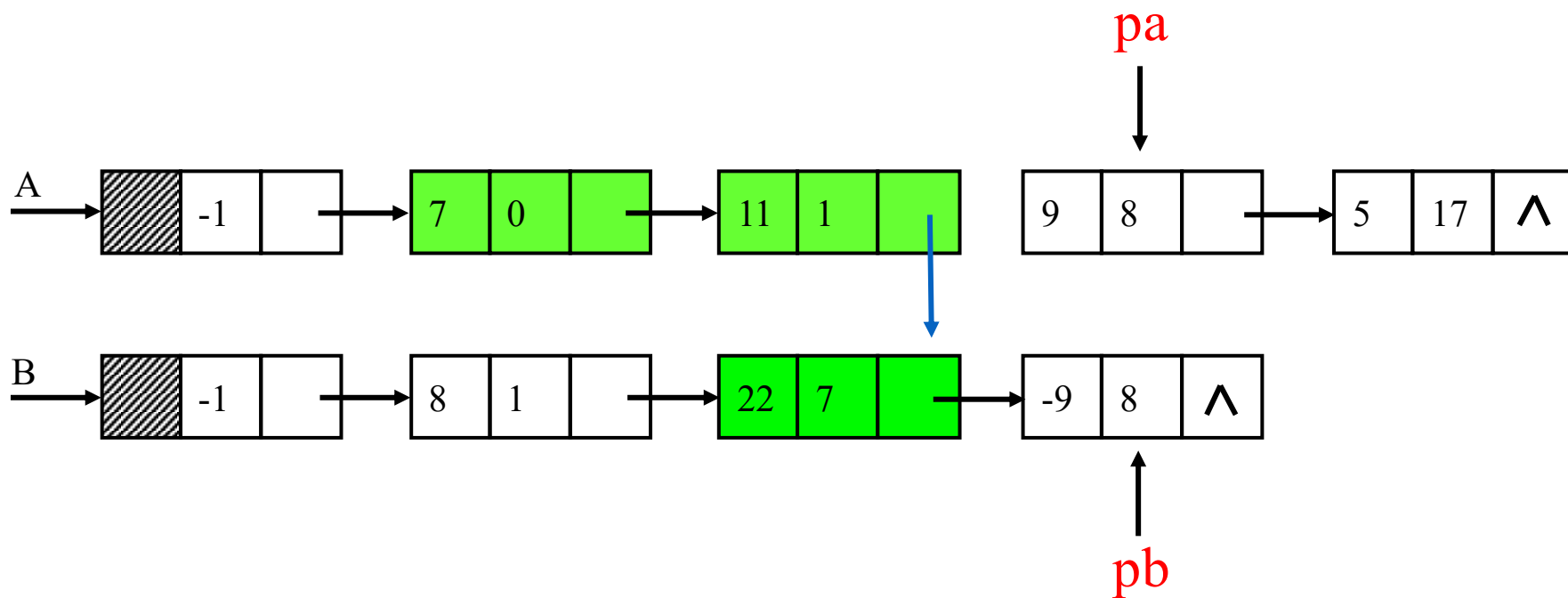
$$B_8(x) = 8x + 22x^7 - 9x^8$$



多项式相加

$$A_{17}(x)=7+3x+9x^8+5x^{17}$$

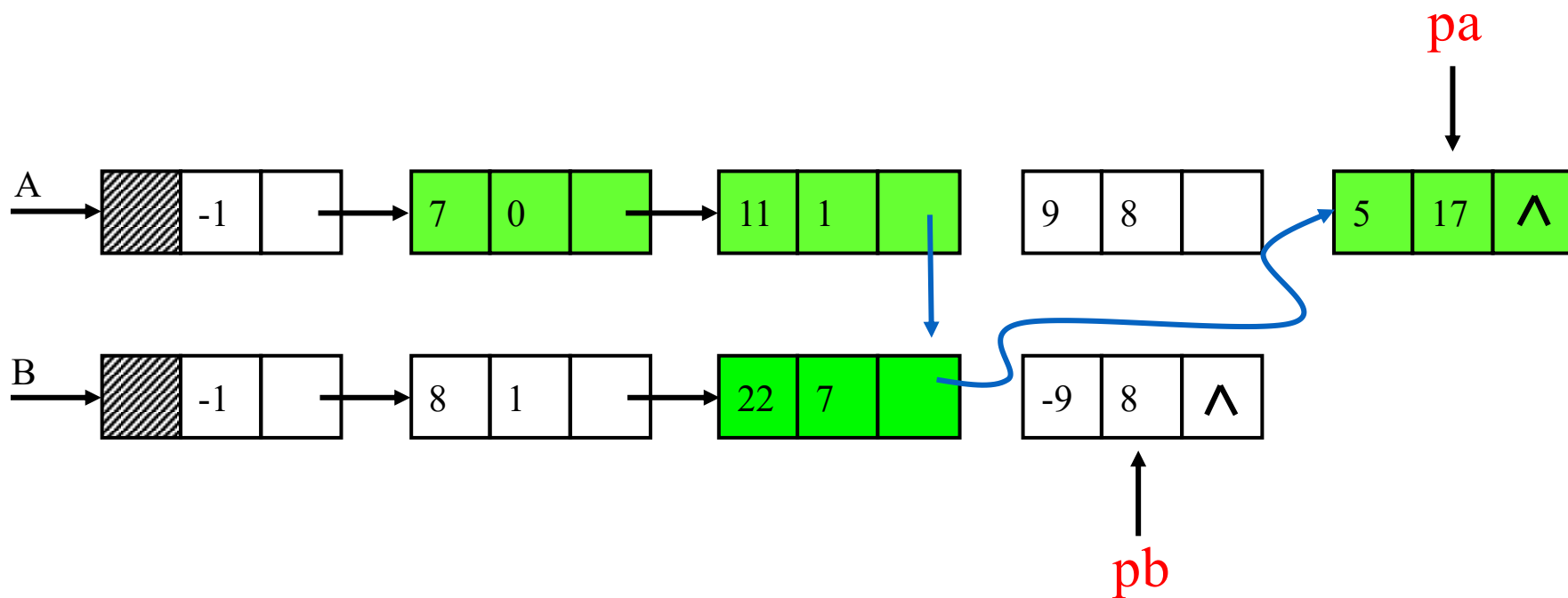
$$B_8(x)=8x+22x^7-9x^8$$



多项式相加

$$A_{17}(x) = 7 + 3x + 9x^8 + 5x^{17}$$

$$B_8(x) = 8x + 22x^7 - 9x^8$$



多项式的表示及其运算

方法1:顺序存储结构直接表示

方法2:顺序存储结构表示非零项

方法3:链表结构存储非零项

启示:

- ✓ 同一个问题可以有不同的表示（存储）方法
- ✓ 有一类共性问题：有序线性序列的组织和管理

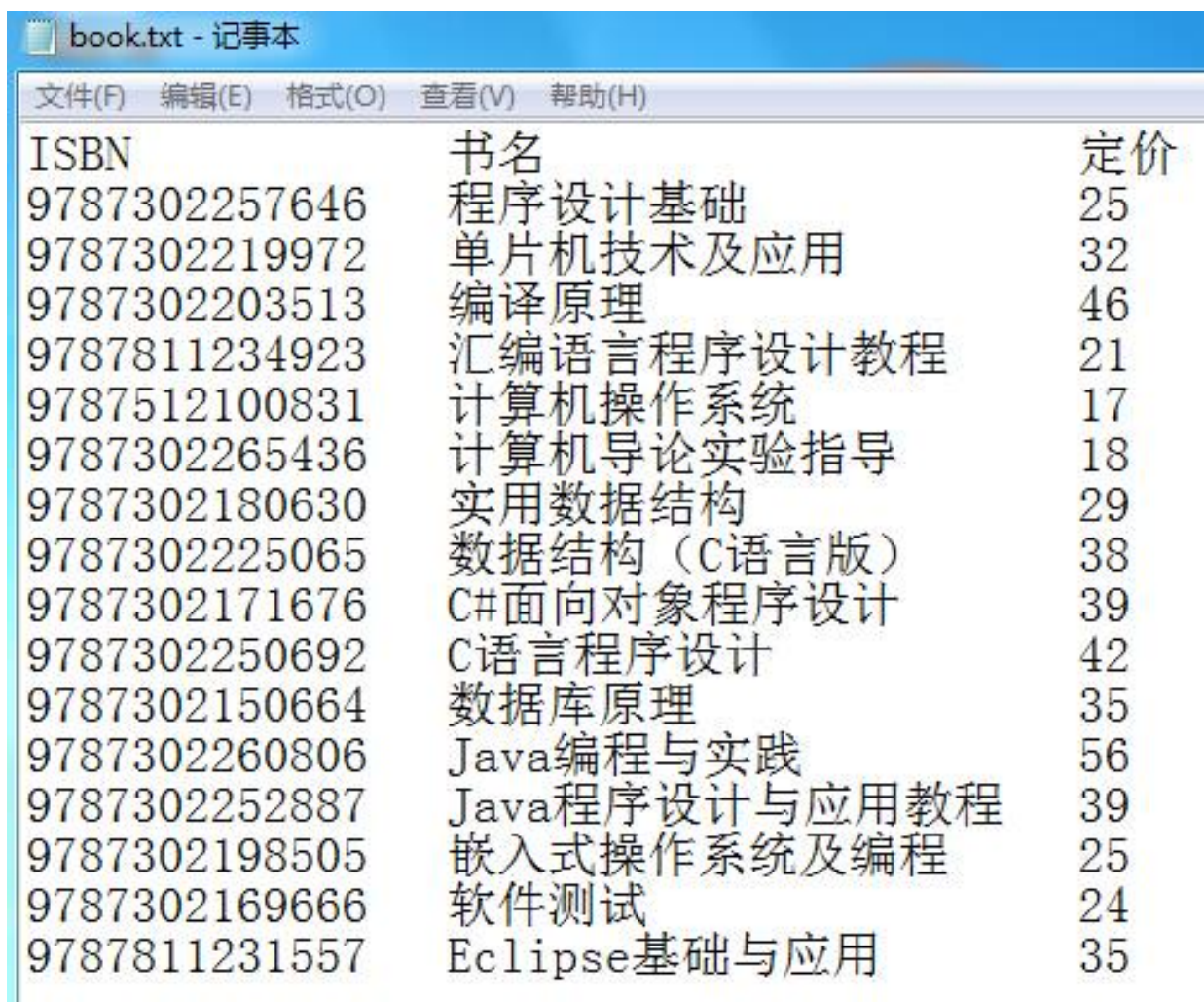
归结为线性表问题

案例2.3：图书信息管理系统

需要的功能：

- (1) 查找
- (2) 插入
- (3) 删除
- (4) 修改
- (5) 排序
- (6) 计数

- 图书表抽象为线性表；
- 表中每本图书抽象为线性表中的数据元素；



The screenshot shows a Notepad window with the title 'book.txt - 记事本'. The menu bar includes '文件(F)', '编辑(E)', '格式(O)', '查看(V)', and '帮助(H)'. The text content is a table with three columns: ISBN, 书名 (Book Title), and 定价 (Price). The table lists 15 books with their respective ISBNs, titles, and prices.

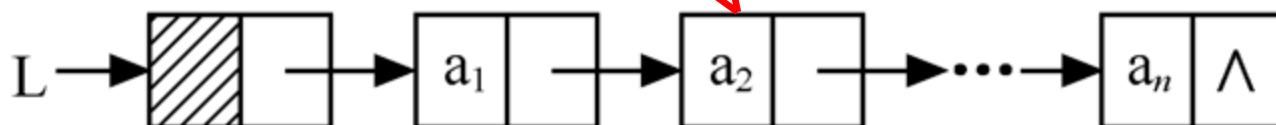
ISBN	书名	定价
9787302257646	程序设计基础	25
9787302219972	单片机技术及应用	32
9787302203513	编译原理	46
9787811234923	汇编语言程序设计教程	21
9787512100831	计算机操作系统	17
9787302265436	计算机导论实验指导	18
9787302180630	实用数据结构	29
9787302225065	数据结构 (C语言版)	38
9787302171676	C#面向对象程序设计	39
9787302250692	C语言程序设计	42
9787302150664	数据库原理	35
9787302260806	Java编程与实践	56
9787302252887	Java程序设计与应用教程	39
9787302198505	嵌入式操作系统及编程	25
9787302169666	软件测试	24
9787811231557	Eclipse基础与应用	35

图书顺序表

elem[0]	elem[1]	elem[2]	...	elem[length-1]	空闲区	
a_1	a_2	a_3	...	a_{length}		



图书链表





- 线性表中数据元素的类型可以为简单类型，也可以为复杂类型。
- 许多实际应用问题所涉的基本操作有很大相似性，不应为每个具体应用单独编写一个程序。
- 从具体应用中抽象出共性的逻辑结构和基本操作（抽象数据类型），然后实现其存储结构和基本操作。



2.3 线性表的类型定义

ADT List {

数据对象:

$D = \{ a_i \mid 1 \leq i \leq n, n \geq 0, a_i \text{ 为 ElemType 类型} \}$

数据关系:

$R = \{ \langle a_i, a_{i+1} \rangle \mid a_i, a_{i+1} \in D, 1 \leq i \leq n-1 \}$

基本运算:

InitList(&L): 构造一个空的线性表。

DestroyList(&L): 销毁线性表。

ListInsert(&L, i, e): 插入元素。

ListDelete(&L, i, &e): 删除元素。

... ..

} ADT List

线性表的存储结构

在计算机内，线性表有两种基本的存储结构：

顺序存储结构和链式存储结构

接下来我们分别讨论这两种存储结构，以及对应存储结构下实现各操作的算法

2.4 线性表的顺序表示和实现



线性表的顺序表示又称为顺序存储结构或顺序映像。

顺序存储定义：把逻辑上相邻的数据元素存储在物理上相邻的存储单元中的存储结构。

简言之，逻辑上相邻，物理上也相邻

顺序存储方法：用一组地址连续的存储单元依次存储线性表的元素，可通过数组 $V[n]$ 来实现。

例如：线性表（1， 2， 3， 4， 5， 6）的存储结构

1	2	3	4	5	6
---	---	---	---	---	---

- 依次存储，地址连续——中间没有空出存储单元。
- 是一个线性表顺序存储结构。

例如：线性表（1， 2， 3， 4， 5， 6）的存储结构

1	2	3	4	5	6
---	---	---	---	---	---

- 依次存储，地址连续——中间没有空出存储单元。
- 是一个线性表顺序存储结构。

1	2	3	4			5	6
---	---	---	---	--	--	---	---

- 地址不连续——中间存在空的存储单元。
- 不是一个线性表顺序存储结构。

线性表的顺序存储结构占用一片连续的存储单元，知道某个元素的存储位置，就可以计算其他元素的存储位置。

顺序表中元素存储位置计算

顺序表的优点：
随机存取

存储地址	存储内容
起始地址/基地址 L_0	元素1
L_0+m	元素2
	
$L_0+(i-1)*m$	元素i
	
$L_0+ (n-1)*m$	元素n

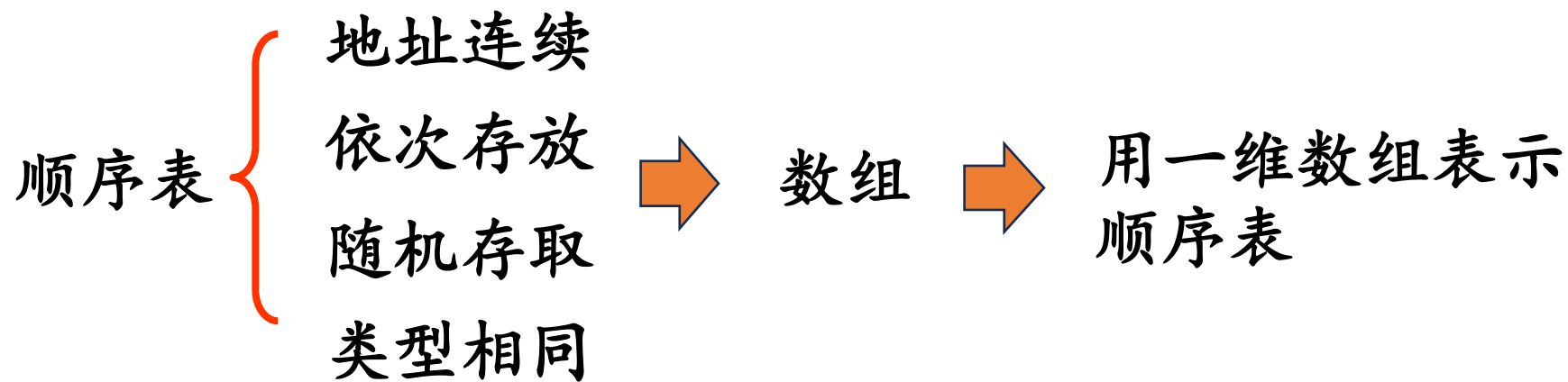
$m=\text{sizeof}(\text{ElemType})$
每个元素占用的存储单元

$$\text{Loc}(\text{元素}i) = L_{i-1} + m$$

$$\text{Loc}(\text{元素}i) = L_0 + (i-1)*m$$

顺序表的结构体数据类型

- 在C语言中，如何表示顺序表？



线性表长度可变（删除）
数组长度不可动态定义 → 用一个额外的变量表示顺序表的长度

```
graph LR; A[线性表长度可变（删除）  
数组长度不可动态定义] --> B[用一个额外的变量表示顺序表的长度];
```

顺序表的结构体数据类型

```
#define MAXSIZE 100    //最大长度
typedef struct {
    ElemType elem[MAXSIZE]; //存放数据元素，数组静态分配
    // ElemType *elem; //存储空间的基地址，数组动态分配
    int length;           //顺序表的当前长度
} SqList; //定义了结构体数据类型SqList，用于表示顺序表
```

- SqList是一个数据类型，含有两个成员
- ElemType是抽象表示，在上机时要注意

图书表的顺序存储结构类型定义

```
#define MAXSIZE 10000    //图书表可能达到的最大长度
typedef struct           //图书信息定义
{
    char no[20];          //图书ISBN
    char name[50];        //图书名字
    float price;          //图书价格
}Book;

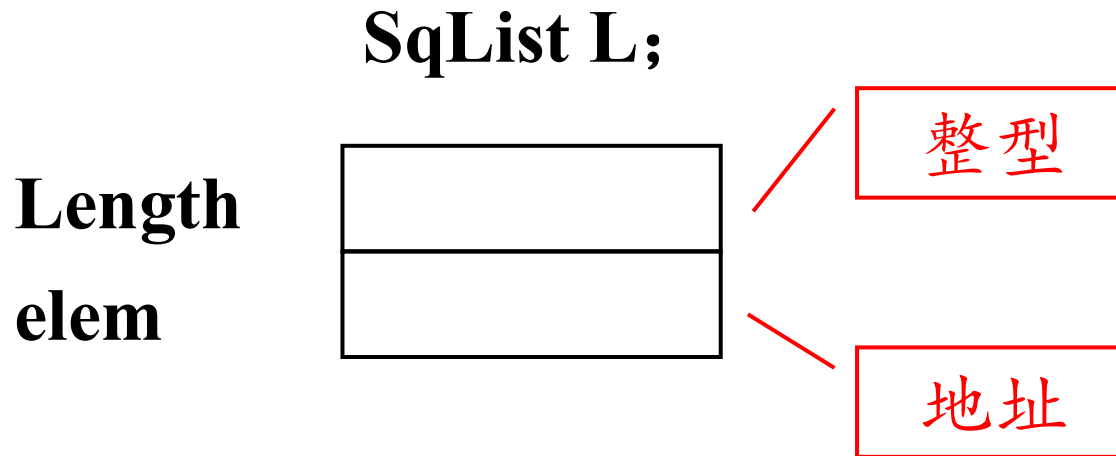
typedef struct
{
    Book *elem;           //存储空间的基地址
    int length;           //图书表中当前图书个数
}SqList;                 //图书表的顺序存储结构类型为SqList
```

- 在上述定义后，可以通过变量定义语句：

`SqList L;` //将L定义为SqList类型的变量，L是个顺序表

- 可通过L.data[i-1]访问表中位置序号为i的图书记录。

注意：L.Length表示顺序表中当前数据元素的个数。C语言中数组下标是从0开始的，而位置序号是从1开始的，要注意区分元素的位置序号和元素在数组中的下标位置，数据元素a1, a2, ..., an依次存放在数组L.elem[0], L.elem[1], ..., L.elem[L.Length-1]中。

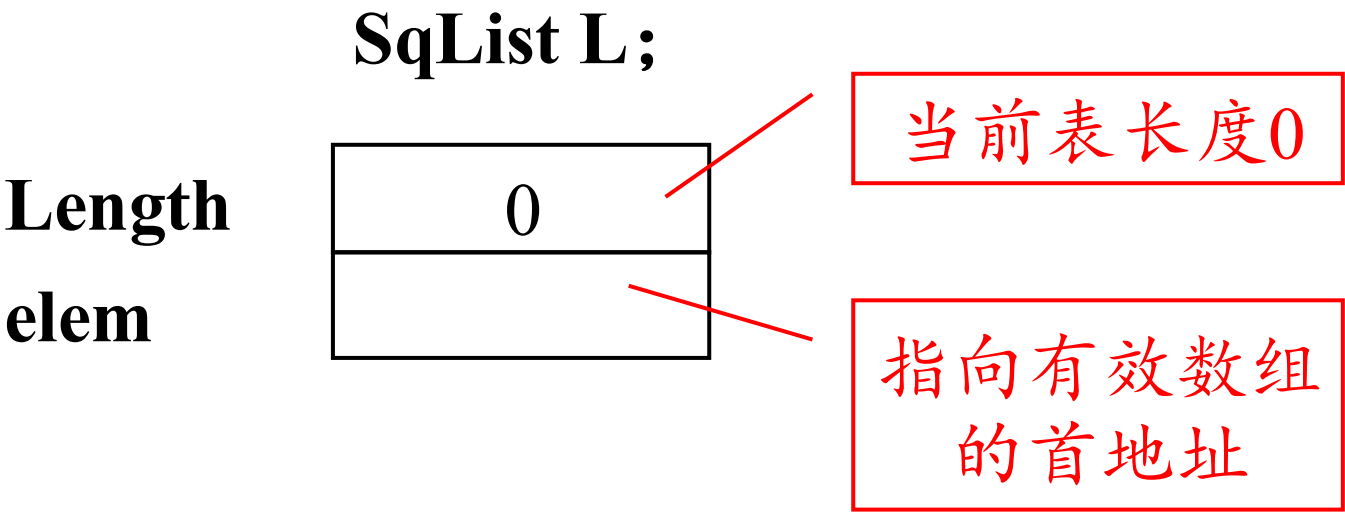


初始状态? 初始L.length与L.elem中是垃圾数据

重要基本操作的算法实现

1. 初始化线性表L（参数用引用，C++）

```
Status InitList_Sq(SqList &L){ //构造一个空的顺序表L
    L.elem=new ElemType[MAXSIZE]; //为顺序表分配空间
    if(!L.elem) exit(OVERFLOW);    //存储分配失败
    L.length=0;                      //空表长度为0
    return OK;
}
```



重要基本操作的算法实现

1. 初始化线性表L（参数用指针, C++）

```
Status InitList_Sq(SqList *L){ //构造一个空的顺序表L
    L-> elem=new ElemType[MAXSIZE]; //为顺序表分配空间
    if(! L-> elem) exit(OVERFLOW);    //存储分配失败
    L-> length=0;                      //空表长度为0
    return OK;
}
```

什么情况下会出现存储分配失败？

重要基本操作的算法实现

2. 销毁线性表L (C++)

```
void DestroyList(SqList &L)
{
    if (L.elem) delete L.elem;    //释放存储空间
}
```

3. 清空线性表L

```
void ClearList(SqList &L)
{
    L.length=0;                //将线性表的长度置为0
}
```

重要基本操作的算法实现

4. 求线性表L的长度

```
int GetLength(SqList L)
{
    return (L.length);
}
```

5. 判断线性表L是否为空

```
int IsEmpty(SqList L)
{
    if (L.length==0) return 1;
    else return 0;
}
```

6. 取值 (根据位置i获取相应位置数据元素的内容)

获取线性表L中的某个数据元素的内容

```
int GetElem(SqList L, int i, ElemType &e)
```

```
{
```

```
    if (i<1||i>L.length) return ERROR;
```

```
    //判断i值是否合理，若不合理，返回ERROR
```

```
    e=L.elem[i-1]; //第i-1的单元存储着第i个数据
```

```
    return OK;
```

```
}
```

$O(1)$

随机存取

可直接通过数组下标定位元素